Zadanie 2 – Mini ETL PRO: Logowanie, abstrakcja i obsługa błędów

Cel zadania:
Rozszerzyć istniejący projekt Mini-ETL (przetwarzanie treningów) o elementy typowe dla systemów produkcyjnych:
• sterowanie logowaniem
• abstrakcję źródeł danych
• metody klasowe jako fabryki obiektów
• obsługę błędów domenowych
• zapis raportu z przetwarzania ETL

Projekt powinien działać na danych w formacie:
date|athlete|type|distance_km|time_min
np.
2026-03-01|Marcin|run|12.5|65

Część A – Abstrakcja źródła danych

1. Stwórz abstrakcyjną klasę ActivitySource wykorzystując moduł abc.
2. Klasa powinna zawierać metodę abstrakcyjną:

```
read_lines() -> Iterator[str]
```

3. Zaimplementuj dwie klasy dziedziczące:

TextFileSource(path)
- czyta linie z pliku tekstowego

InMemorySource(lines)
- przyjmuje listę linii (do testów)

Dla chętnych:

CSVSource(path) lub HttpSource(url).

Część B – Metody klasowe

W klasie TrainingLog dodaj metodę klasową:

```
@classmethod
def from_source(cls, source: ActivitySource, *, strict: bool = False)
```

Metoda powinna:
• pobierać linie ze źródła
• parsować je do obiektów Activity
• tworzyć obiekt TrainingLog

Tryby pracy:
strict=False
- błędne linie są pomijane
- zapisywany jest log ostrzegawczy

strict=True
- przy pierwszym błędzie przetwarzanie zostaje przerwane.

Część C – Logowanie

Stwórz dataclass LoggingConfig z polami:

level
to_file
log_file
rotate (opcjonalnie)
max_bytes
backup_count

Zaimplementuj funkcję:

```
configure_logging(config: LoggingConfig)
```

Funkcja powinna skonfigurować logger Python (moduł logging).

Wymagania:
• logi na konsoli
• opcjonalnie logi do pliku
• poziomy DEBUG / INFO / WARNING / ERROR

Część D – Obsługa błędów

Zdefiniuj hierarchię wyjątków:

class ETLError(Exception)
class ParseError(ETLError)
class SourceError(ETLError)
class ValidationError(ETLError)

Parser powinien zgłaszać:
ParseError – gdy linia ma zły format
ValidationError – gdy wartości są niepoprawne
SourceError – gdy nie można odczytać danych

W trybie non-strict:
• błędy są logowane

W trybie strict:
• program przerywa działanie.

Część E – Raport ETL

Po zakończeniu przetwarzania program powinien zapisać raport JSON zawierający:

started_at
finished_at
source
strict
processed_lines
accepted_records
rejected_records
errors

Każdy błąd powinien zawierać:
line_no
line
error_type
message

Część F – Struktura projektu

Zalecana struktura:

activity.py
sources.py
etl_errors.py
logging_config.py
pipeline.py
training_log.py
reporting.py
main.py